



S. G. Ganesh

Levels of Exception Safety

The concept of 'exception safety' is important for programming in the presence of exceptions. In this article, we'll look at different levels of exception safety, with the help of an example.

Let's first look at an actual code example to understand why 'exception safety' is important. The following Java code is from Axion DB ver 1.1, from the file `axion/whiteboard/one/src/org/axiondb/util/BTree.java`:

```
public void read() throws IOException {
    // ...
    FileInputStream fin = new FileInputStream(_idxFile);
    ObjectInputStream in = new ObjectInputStream(fin);
    _leaf = in.readBoolean();
    int size = in.readInt();
    for (int i = 0; i < size; i++) {
        _keys.addInt(in.readInt());
        _vals.addInt(in.readInt());
    }
    in.close();
    fin.close();
}
```

Can you find out what can go wrong in this code? Here, the `close()` method will not be called if any exception is thrown after `fin` and `in` are initialised, inside the `for` loop. The Java idiom is to put them in *finally* blocks, to ensure that `close()` statements are called even if an exception has occurred, in order to avoid resource leaks. In other words, this code has no exception safety.

A method can have four levels of 'exception safety':

- No exception safety: There is no guarantee on the effect of throwing an exception. Because of the exception, resources might leak, and/or the underlying object can be in a corrupted state.
- Basic exception safety: No resources are leaked. The operation might have caused some side-effects, but the object is in a consistent state (i.e., invariants are preserved). The state of the object might have changed.

- Strong exception safety: No resources are leaked. The operation might either completely fail or fully succeed, but not be partially complete. In other words, commit or roll-back semantics are implemented.
- No-throw exception safety: Operations are guaranteed to succeed, and no exceptions will be thrown.

This concept is language-independent, and hence applies to languages like C++, Java and C#. Since 'safety' here means the level of assurance given by a method, it is also known as a 'guarantee'.

No exception guarantee means that the function is really unsafe in the presence of exceptions, and that such a function can lead to resource leaks, or can corrupt the objects it manipulates. A basic exception guarantee only means that it will not leak resources, whether an exception has occurred or not. Still, a basic exception guarantee can leave objects in a partially changed (but consistent) state. A strong exception guarantee's commit or roll-back semantics (as in transaction processing in database-management systems) ensures an operation is either fully executed or not executed, but never partially executed. Hence, in practice, this behaviour is desirable for the programs that we write. It is not practically feasible to always guarantee that methods will never throw any exception, or that they will always succeed (and never fail). However, that is possible in some cases, such as an implementation of a swap method, which exchanges two variables, and will never throw an exception.

Let us look at an example to understand these levels of guarantees. Assume that you're implementing an application that manipulates huge text files. One functionality it provides is removing duplicate lines in the text files. Assume that this functionality is implemented using a method named `removeDuplicates`, and it takes a file handle as